

Few-shot Learning on AMS Circuits and Its Application to Parasitic Capacitance Prediction

Shan Shen^{1,2}, Yibin Zhang², Hector Rodriguez Rodriguez², Wenjian Yu²

¹*School of Microelectronics, Nanjing University of Science and Technology, Nanjing 210094, China*

²*Dept. Computer Science & Tech., BNRist, Tsinghua Univ., Beijing 100084, China*

Email: shanshen@njust.edu.cn, yu-wj@tsinghua.edu.cn

Abstract—Graph representation learning is a powerful method to extract features from graph-structured data, such as analog/mixed-signal (AMS) circuits. However, training deep learning models for AMS designs is severely limited by the scarcity of integrated circuit design data. In this work, we present CircuitGPS, a few-shot learning method for parasitic effect prediction in AMS circuits. The circuit netlist is represented as a heterogeneous graph, with the coupling capacitance modeled as a link. CircuitGPS is pre-trained on link prediction and fine-tuned on edge regression. The proposed method starts with a small-hop sampling technique that converts a link or a node into a subgraph. Then, the subgraph embeddings are learned with a hybrid graph Transformer. Additionally, CircuitGPS integrates a low-cost positional encoding that summarizes the positional and structural information of the sampled subgraph. CircuitGPS improves the accuracy of coupling existence by at least 20% and reduces the MAE of capacitance estimation by at least 0.067 compared to existing methods. Our method demonstrates strong inherent scalability, enabling direct application to diverse AMS circuit designs through zero-shot learning. Furthermore, the ablation studies provide valuable insights into graph models for representation learning.

I. INTRODUCTION

Deep learning (DL) models rely on the availability of large-scale labeled datasets to train deep neural networks with millions of parameters. In many real-world scenarios, acquiring such extensive labeled datasets is impractical or impossible. For example, obtaining practical and real integrated circuit (IC) designs from the industry is notoriously difficult due to a combination of intellectual property concerns, security risks, and the complexity of the designs. IC designs are the result of extensive research and substantial investment. Strict intellectual property (IP) laws prevent the sharing of proprietary designs without proper authorization. Some IC designs are critical for defense and infrastructure, making them sensitive from a security perspective. Another reason for data scarcity in the IC field is the high cost of labeling data for tasks such as rare event prediction, layout optimization, or parasitic extraction. These tasks demand expert knowledge and the use of time-consuming EDA tools.

Consequently, there is a growing need for models that can learn effectively from limited amounts of design data.

This work is supported by the National Natural Science Foundation of China (NSFC) under grant (No. 62204141) and Beijing Natural Science Foundation (No. Z230002). W. Yu is the corresponding author. The code and the small datasets are available at <https://github.com/ShenShan123/CirGPS.git>

Few-shot learning (FSL) addresses this challenge by enabling models to generalize to new tasks or classes using only a small number of labeled examples [1]. Inspired by the human ability to learn new concepts from minimal information, FSL reduces the dependency on large datasets to make machine learning models more adaptable and efficient. This paradigm shows great potential in the EDA field [2].

In this work, we leverage several FSL techniques in the application of analog/mixed-signal (AMS) circuit design automation. AMS circuit design often demands substantial manual intervention. It relies heavily on manually selecting circuit topology and component sizes. In conventional AMS design workflows, circuit designers rely on pre-layout simulations for design optimization, followed by post-layout verification. However, as process technologies scale to smaller transistors and lower operating voltages, the reduced transistor driving capability exacerbates sensitivity to parasitic effects. For example, the coupling capacitance has become too significant to be overlooked in simulations, producing a substantial disparity between pre-layout and post-layout performance metrics [3], [4].

To address the challenges described above, we present a general, powerful, scalable circuit graph learning method (**CircuitGPS**) to perform various parasitic-related tasks in AMS circuits. The circuits are modeled as heterogeneous graphs, while the coupling effects are treated as links to be predicted. Inspired by SEAL [5], a small-hop subgraph sampling technique is proposed to sample target links from a large circuit graph. Then, we customize a graph Transformer (GT) [6] to extract the meta-embeddings at the subgraph level. The pre-trained meta-learner, combined with its learned embeddings, is subsequently adapted for the downstream task of coupling capacitance regression. Our major contributions are summarized as follows:

- A small-hop subgraph sampling technique is proposed to convert a link/node into a subgraph. This universal and scalable operation enables FSL by decoupling the target links/nodes from the specific circuit graph. Therefore, we can leverage more powerful and sophisticated DL models aimed at graph-level representation learning.
- A low-cost position encoding, double-anchor shortest path distance (**DSPD**), is defined to measure the relative position w.r.t the anchors of the subgraph. DSPD improves the model expressivity and leads to better

generalization.

- A powerful framework, **CircuitGPS**, is built on the top of the GraphGPS framework [6], which can perform multiple tasks on large circuit graphs. The framework was complemented with a fine-tuning strategy for the downstream capacitance regression.
- Through our ablation studies, we summarize two key insights into DL graph models trained on the AMS circuits: (1) involving the original node features during model training for link prediction will reduce generalization of the graph model; (2) classic message-passing graph neural networks (MPNNs) can achieve highly competitive performance, even outperforming GTs.

II. BACKGROUND

Few-shot learning. FSL can be formally defined as a machine learning problem where a model must quickly adapt to a new task using only a few training examples or “shots” per class [7]. Typically, FSL involves a “meta-learning” phase, where the model learns how to learn across a variety of tasks, and an “adaptation” phase, where it applies this knowledge to a previously unseen task with limited data.

Subgraph sampling. Zhang and Chen [8] first extracted local enclosing subgraphs around links as the training data, and used a fully-connected neural network to learn which enclosing subgraphs correspond to link existence, which is called Weisfeiler-Lehman Neural Machine. The enclosing subgraph for a node pair (m, n) is the subgraph induced from the network by the union of the neighbors of m and n up to h hops. These enclosing subgraphs are very informative for link prediction – all first-order heuristics such as common neighbors can be directly calculated from the 1-hop enclosing subgraphs. However, it is shown that high-order heuristics often have much better performance than first and second-order ones [9], but the time and memory consumption are unsuitable for most practical applications. In work [5], Zhang et al. found that most high-order heuristics can be unified by a γ -decaying theory, and that even a small h can safely learn good high-order features.

Graph Transformers (GT). Considering the great successes of Transformers in natural language processing [10] and also in computer vision [11], it is natural to study their applicability in the graph domain. A fully-connected GT [12] was first introduced by Dwivedi et al. SAN [13] further improved this work by implementing invariant aggregation of the positional encoding (PE) Laplacian eigenvectors and conditional attention for the real and virtual edges of a graph. Concurrently, Graphormer [14] used pair-wise graph distances (or 3D distances) to define relative positional encodings, with outstanding success on large molecular benchmarks.

Positional and structural encodings. There have been many recent works on PE and structural encoding (SE), notably on Laplacian PE (LapPE) [12], [13], shortest-path-distance (SPD) [14], node degree centrality [14], random-walk SE (RWSE) [15], and more. Some works also proposed

dedicated networks to learn the PE/SE from an initial encoding [13], [15]. To better understand the different PE/SEs and the contribution of each work, Rampasek et al. [6] categorized them into different types and examined their effect. In most cases, PE/SE serves as a soft bias, meaning they are simply provided as input features. However, in some cases, they can be used to direct messages [16] or create bridges between distant nodes [17].

Parasitic prediction. The DL models are gaining increasing attention and are being leveraged for predicting parasitic effects in AMS circuits. ParaGraph [18] converts circuit schematics into graphs and utilizes different types of MPNN layers to predict net capacitance and device layout parameters. It includes the ensemble modeling technique to improve the prediction accuracy via training three different sub-models to predict different capacitance magnitudes. In work [19], Shen et al. developed a DL-based multi-expert model (DLPL-Cap) to predict parasitic capacitance in the pre-layout stage. The model combines a GNN router and five expert regressors, effectively managing the imbalance of the net parasitic capacitance in SRAM circuits. They achieved state-of-the-art results on capacitance regression.

However, the aforementioned research [18], [19] treats the coupling capacitance as a part of the lumped-in capacitance. Furthermore, these ensemble models have very limited generalization and transferability when making predictions on unseen designs, as they do not employ FSL techniques (see Section IV for more details).

III. METHODS

The proposed method contains four steps, as shown in Fig. 2: (1) AMS netlist conversion, (2) enclosing subgraph extraction, (3) position encoding, and (4) model pre-training/fine-tuning. CircuitGPS is first pre-trained on a large dataset for link prediction, taking local enclosing subgraphs around links as the input, and outputting the link existence likelihood. After the pre-training, we can obtain a “meta-learner”, which can be directly tested on other unseen circuits with zero-shot or be further used as an initial model for other downstream tasks. For example, for the latter usage, we can select a regression task where the model predicts the coupling capacitance between the given nodes. This demonstrates that the meta-learner can be quickly adapted to other tasks.

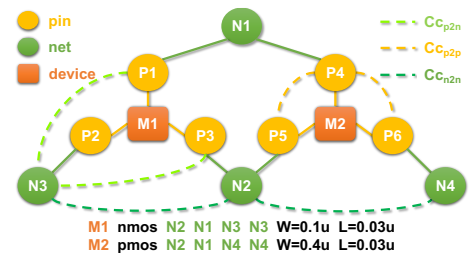


Fig. 1 A buffer netlist is converted to a heterogeneous graph composed of net, pin, and device nodes, where coupling effects are modeled as target links (dash lines).

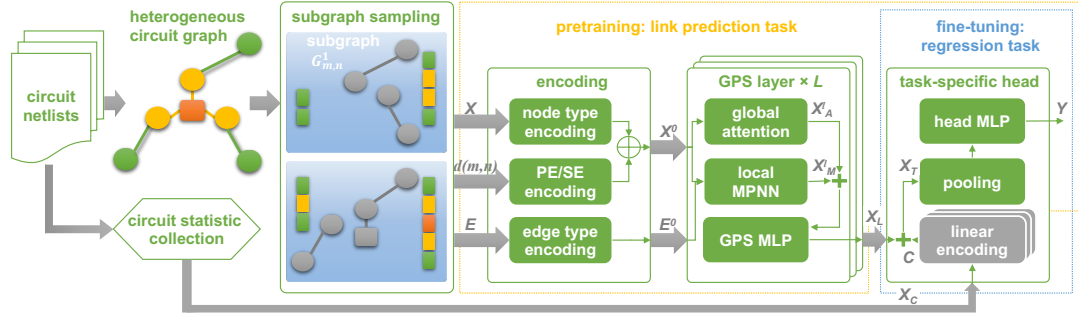


Fig. 2 **CircuitGPS** is based on a parallel MPNN+Transformer layer. The overall workflow includes: AMS netlist conversion, enclosing subgraph extraction, position encoding, and model pre-training/fine-tuning.

A. AMS Netlist Graph

The schematic netlist of an entire AMS circuit is modeled as a large heterogeneous graph $G = (V, E)$ with node attribute $\mathbf{X} = \{x_0, x_1, \dots, x_{N-1}\}^T$ and edge attribute $\mathbf{E} = \{e_0, e_1, \dots, e_{N_E-1}\}^T$. Fig. 1 shows an example of a buffer circuit containing three types of nodes. Green circles represent nets with $x_i = 0$, connecting to devices; orange squares represent instances of the transistor device with $x_i = 1$, which may also represent other devices, such as capacitors, resistors, or diodes; yellow circles represent pins of devices with $x_i = 2$. Edges describe the topology of a schematic, which can be “device-to-pin” connections with $e_i = 0$ or “net-to-pin” connections with $e_i = 1$. Links are the targets to be predicted, which are composed of “pin-to-net” coupling with $e_i = 2$, “pin-to-pin” coupling with $e_i = 3$, and “net-to-net” coupling with $e_i = 4$. Note that the terminology “link” is different from “edge” in this work, as “link” is our ground truth that can only be extracted from the post-layout netlist. We set all the edges and links to be undirected.

B. Subgraph Sampling

Here we describe our enclosing subgraph sampling technique. The sampling does not restrict the learned features to be in some particular form but instead learns general graph-level structural features for both node- and link-level targets. The sampling procedure contains three steps: (1) negative link generation, (2) enclosing subgraph extraction, and (3) node/edge attribute matrix construction.

Negative link generation. For a graph $G = (V, E)$ with an $N \times N$ adjacency matrix $\mathbf{A}$, a positive link (observed) exists from node i to node j if $\mathbf{A}_{i,j} = 1$, while a negative link (unobserved) corresponds to $\mathbf{A}_{i,j} = 0$ or $\mathbf{A}_{i,j} = -1$. Suppose that a graph has nodes $V = \{i, j, m, n\}$ and edges $E = \{(i, j), (m, n)\}$, where i, m are the source and j, n are the destination. For each type of positive link, the structural negative links are formed by permuting the source/destination nodes in E , resulting in $E_N = \{(i, n), (m, j)\}$. The negative and positive links have the same edge type, which means the sources/destinations of these links also share the same node types. In this case, $x_i = x_m$ and $x_j = x_n$.

In AMS circuits, a positive link refers to the presence of a coupling effect between the net/pin node pair, while a

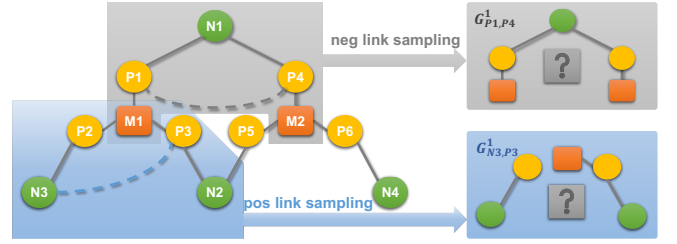


Fig. 3 One-hop enclosing subgraph sampling.

negative link indicates the absence of such an effect. The positive and negative links are assigned the labels ‘1’ and ‘0’, respectively. As depicted in Fig. 1, the target links have three types. We found that the “pin-to-net” links E^{p2n} often constitute the majority, whereas “net-to-net” links E^{n2n} are relatively fewer. The severe class imbalance can degrade the pre-trained model’s performance [19]. To mitigate this, we balance the training dataset by randomly sampling $|E^{n2n}|$ instances from each link type.

Enclosing subgraph extraction. Here we follow the definition of enclosing subgraph in SEAL [5].

Definition 1: (Enclosing subgraph) For a graph $G = (V, E)$, given two nodes $m, n \in V$, the h -hop enclosing subgraph for (m, n) is the subgraph $G_{m,n}^h$ induced from G by the set of nodes $\{i \mid d(i, m) \leq h \cup d(i, n) \leq h\}$.

The enclosing subgraph describes the “ h -hop surrounding environment” of (m, n) . $G_{m,n}^h$ contains all h -hop neighbors of m and n . Since the circuit graphs could be extremely large (see Table IV for details), we found that the 1-hop sampling achieves the best balance between performance and efficiency. This choice is also justified by the γ -decaying theory in SEAL, which supports the use of a small h to learn good high-order features. Figure 3 shows an example of the extracted subgraph $G_{m,n}^1$.

Attribute matrix construction. A MPNN typically takes $(\mathbf{A}, \mathbf{X})$ as input. Due to the heterogeneity of subgraphs, the node types form an attribute matrix $\mathbf{X} \in \{0, 1, 2\}^{N \times 1}$. Then, $\mathbf{X}$ is fed to the node-type encoder of CircuitGPS to learn embeddings $\text{Embed}(\mathbf{X}) \in \mathbb{R}^{N \times d_0}$. The model also considers edge attributes, where another edge-type encoder generates edge embeddings $\text{Embed}(\mathbf{E}) \in \mathbb{R}^{N_E \times d_0}$.

The aforementioned attributes are critical for pre-training

TABLE I Dimension Definition in $\mathbf{X}_C$.

Node Type	Dim.	Definition
$x_i = 0$ for net node	0	# of connected transistors
	1	# of connected gate terminals
	2	# of connected source/drain terminals
	3	# of connected base terminals
	4	Total width of connected transistor
	5	Total length of connected transistor
	6	# of connected capacitors
	7	Total length of connected capacitors
	8	Total # of connected capacitor fingers
	9	# of connected resistors
	10	Total width of connected resistors
	11	Total length of connected resistors
	12	# of connected ports
$x_i = 1$ for device node	0	Multiplier of transistors
	1	Length of the transistor
	2	Width of the transistor
	3	Multiplier of connected resistors
	4	Length of resistor
	5	Width of resistor
	6	Multiplier of connected capacitor
	7	Length of capacitor
	8	# of capacitor fingers
	9	# of ports in the device instance
	10	Type code of the device instance
$x_i = 2$ for pin node	0	Pin types (G/D/S/B for MOS)

a link prediction model. However, for the downstream regression, the coupling capacitance is also related to other information, such as the geometric size of devices and the number of devices connecting to a net. Thus, we collect more detailed statistics for each type of node, including the design parameters in AMS circuits. We pad these statistics into another node attribute matrix $\mathbf{X}_C \in \mathbb{R}^{N \times d_C}$ that is input into the task-specific head of CircuitGPS. Table I lists the definition of each dimension in $\mathbf{X}_C$ for different node types.

C. Positional Encoding

Positional encodings (PEs) are proposed to estimate the position in space of a given node within the graph. Hence, when two nodes are close to each other within a graph, their PE should also be close. A common approach is to compute the pair-wise distance between each pair of nodes or their eigenvectors in the entire graph as proposed in [13], [14], but this is not compatible with linear Transformers (such as Performer [20]) as it requires to materialize the entire attention matrix. A better fitting solution is to use the eigenvectors of the graph Laplacian or their gradient [13], [16], [21]. However, calculating those PEs is both time- and resource-consuming due to the large scale of subgraphs sampled from AMS circuits. For this reason, a more computationally efficient PE is demanded by our application.

The proposed PE, named double anchor shortest path distance (DSPD), is a relative PE according to the PE/SE categories proposed in [6], [22]. In each subgraph $G_{m,n}^1$ with anchors m and n , we calculate the shortest path distance ($d(i, m), d(i, n)$) to the two anchors for each node i . The DSPD is inspired by the double-radius node labeling (DRNL) in SEAL, where the authors iteratively assign larger labels to

TABLE II Comparison of Different PEs in Link Prediction.

PE	Acc.	F1	AUC	Time/G (s)
w/o PE	0.8867	0.9120	0.9393	N/A
$\mathbf{X}_C$	0.9066	0.9261	0.9629	N/A
DRNL	0.9505	0.9640	0.9698	0.0170
RWSE	0.8931	0.9255	0.8612	0.1296
LapPE	0.9561	0.9680	0.9697	0.1934
DSPD	0.9618	0.9720	0.9774	0.0173

nodes with a larger distance (radius) with respect to both anchor nodes. The labeling principle is a perfect hashing function that allows fast closed-form computations. Differently, the DSPD encoding does not compute a unique label; it directly stores the distance vector ($d(i, m), d(i, n)$). The PE encoder then uses the distances to generate $\mathbf{D}_0$ and $\mathbf{D}_1$. We concatenate them with the node-type embeddings to form the final node attribute matrix,

$$\mathbf{X}^0 = \mathbf{D}_0 \oplus \mathbf{D}_1 \oplus \text{Embed}(\mathbf{X}), \quad (1)$$

which is the input of the first GPS layer.

Table II shows the prediction accuracy of CircuitGPS on link prediction with different PEs, including circuit statistic matrix $\mathbf{X}_C$, RWSE proposed by [15], LapPE used by [12], [13], DRNL proposed by SEAL, and our DSPD. We take the AMS design SSRAM [23] as a small training dataset and the circuit DIGITAL_CLK_GEN as a test dataset to evaluate CircuitGPS in a zero-shot learning setting. The proposed DSPD demonstrates the highest accuracy while ensuring computational efficiency.

Observation 1: Taking the circuit statistic matrix $\mathbf{X}_C$ as a positional encoding reduces the model performance compared to employing other PEs.

This counterintuitive phenomenon was also reported by Shi et al. [24]. In their work, removing original node features of neighbor nodes led to slight performance gains for SEAL on multiple link prediction datasets. While seemingly paradoxical, this result is explicable: in link prediction tasks, neighbor node features may introduce task-irrelevant noise. For instance, in our context, the specific device types connected to a target net are irrelevant for predicting coupling effects between nodes. This critical observation motivated our architectural redesign, transitioning from GraphGPS to the simplified CircuitGPS framework.

D. Hybrid Graph Transformer

The hybrid GT model used in this work is built from a general, powerful, and scalable architecture named GraphGPS [6]. Each GPS layer is a parallel MPNN+Transformer layer, as shown in Fig. 2. The update function is

$$\mathbf{X}^{\ell+1}, \mathbf{E}^{\ell+1} = \text{GPS}^\ell(\mathbf{X}^\ell, \mathbf{E}^\ell, \mathbf{A}), \quad (2)$$

and the hidden features are computed as

$$\mathbf{X}_M^{\ell+1}, \mathbf{E}^{\ell+1} = \text{MPNN}_e^\ell(\mathbf{X}^\ell, \mathbf{E}^\ell, \mathbf{A}), \quad (3)$$

$$\mathbf{X}_A^{\ell+1} = \text{GlobalAttn}^\ell(\mathbf{X}^\ell), \quad (4)$$

$$\mathbf{X}^{\ell+1} = \text{MLP}^\ell(\mathbf{X}_M^{\ell+1} + \mathbf{X}_A^{\ell+1}), \quad (5)$$

TABLE III Ablation Study of Different GPS Layer Configuration on Link Prediction (**best**, second best).

MPNN	Attention	Acc.	F1	AUC	Time(s)	#Param.
None	Performer	0.9458	0.9602	0.9668	1663.0	762,390
None	Transformer	0.9456	0.9601	0.9187	3490.0	778,833
GatedGCN	Performer	0.9618	0.9720	0.9774	1446.1	752,785
GatedGCN	Transformer	0.97011	0.9780	0.9980	2832.9	540,337
GatedGCN	None	0.9693	0.9775	0.9848	965.6	724,854

where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the adjacency matrix; $\mathbf{X}^\ell \in \mathbb{R}^{N \times d_\ell}$, $\mathbf{E}^\ell \in \mathbb{R}^{E \times d_\ell}$ are the d_ℓ -dimensional node and edge attributes, respectively; MPNN_e^ℓ and GlobalAttn^ℓ are instances of an MPNN with edge attributes and a global attention mechanism, respectively; MLP^ℓ is a 2-layer MLP block. After each functional block (an MPNN layer, a GlobalAttn layer, or an MLP), we apply residual connections followed by batch normalization (BN), which are omitted in (5) for clarity. Edge attributes are only passed to the MPNN layer. Due to the scalability of GPS, both MPNN and GlobalAttn layers are modular, where MPNN can be any function that acts on a local neighborhood and GlobalAttn can be any fully connected layer.

Based on Observation 1, we customize the GraphGPS with a task-specific head. We add two linear projection layers and an embedding layer in the head to project circuit statistics $\mathbf{X}_C$ (Table I) into a unified attribute matrix $\mathbf{C} \in \mathbb{R}^{N \times d_L}$. For a node i in the subgraph with node type x_i and circuit statistic x_C^i , c_i is produced by

$$c_i = \begin{cases} \text{Linear}(x_C^i), & \text{if } x_i = 0; \\ \text{Linear}(x_C^i), & \text{if } x_i = 1; \\ \text{Embed}(x_C^i), & \text{if } x_i = 2. \end{cases} \quad (6)$$

Finally, the hidden input of the task-specific head MLP is

$$\mathbf{X}_H = \text{Pool}(\mathbf{X}^L + \mathbf{C}). \quad (7)$$

To find a better layer architecture, we produced an ablation study of different layer types on a small training set as shown in Table III. The alternatives are a global attention-based Transformer, a linear Transformer (Performer [20]), a pure MPNN (GatedGCN [25]), and their combinations.

Observation 2: Classic MPNN models achieve strong performance, matching or even exceeding that of pure Transformers or MPNN+Transformers.

This observation coincides with the results reported by Luo et al. in their recent work [26]. According to their findings, this strong performance is driven by normalization, dropout, and residual connections. All these features have been integrated into CircuitGPS and can be adjusted with the hyperparameters in configuration files. Beyond excellent performance, MPNN requires lower training overhead and converges more easily.

E. Model Fine-tuning

Models trained with few-shot learning techniques are often better at generalizing to new tasks or classes, as they focus on capturing transferable knowledge rather than memorizing

specific data patterns. The pre-trained CircuitGPS on link prediction aims to learn an informational representation of subgraphs. Fine-tuning can be implemented in two ways: (1) freezing the encoders and GPS layers while training only the task-specific head, which significantly accelerates the convergence, or (2) continuing to train all parameters in CircuitGPS using the pre-trained ones as initialization, further improving edge regression accuracy (Table VI).

IV. EXPERIMENTS

Our implementation is based on PyG [27], GraphGym [28], and GraphGPS [6] modules. All experiments were run in a shared computing cluster. The cluster has 40 Intel Xeon Silver CPUs with 128GB of memory and four Nvidia 4090 (24GB) GPUs. Each experiment was executed using four to six CPUs, a single GPU, and 128 GB of RAM.

Circuit statistics were obtained by converting the netlist to a graph. Before the enclosing subgraph sampling, we followed the setup of SEAL [5], where both the positive edges and the negative edges were injected into the original circuit graph. The sampling procedure was implemented with the multi-processing package to reduce the sampling time overhead. We prepared the full schematic netlists to extract the graph and features, and the post-layout netlists (SPF files) to collect the ground-truth labels and targets. The configuration files for experiments in this section can be found in our [Git repository](#).

A. Dataset

Table IV summarizes the AMS designs used as datasets in this work. All designs are under 28nm CMOS technology. SSRAM [23] is a small design with high energy efficiency, consisting of standard digital cells and static random access memory (SRAM) arrays. ULTRA8T SRAM [29] is a multi-voltage design that contains large analog modules and SRAM arrays. SANDWICH-RAM [30] is evenly divided between digital circuits for computing and SRAM arrays for storage, forming a sandwich-like structure. Due to the large scale of the datasets, training with all coupling links results in unacceptable time overhead. The redundancy of capacitance also leads to inefficient training [19]. Thus, we only sample a fraction of the links to reduce the training time per epoch, as shown in the “#links” column of Table IV.

In terms of test data sets, DIGITAL_CLK_GEN is composed of digital cells and SRAM columns and is used to generate the internal clock in SRAM. It is the most challenging test case of our experiments. TIMING_CONTROL consists of standard digital cells to produce control signals for the SRAM module. ARRAY_128_32 is a single 128-row 32-column SRAM array.

TABLE IV AMS Circuit Dataset Statistics.

Split	Dataset	N	N_E	#Links	$N/G_{m,n}^1$	$N_E/G_{m,n}^1$
Train	SSRAM	87K	134K	131K	153	917
	ULTRA8T	3.5M	13.4M	166K	257	1,476
	SANDWICH-RAM	4.3M	13.3M	154K	472	2,540
Test	DIGITAL_CLK_GEN	17K	36K	4K	417	2,403
	TIMING_CONTROL	18K	44K	5K	59	387
	ARRAY_128_32	144K	352K	110K	150	803

TABLE V Accuracy Comparison of Different Methods on Link Prediction.

Dataset Metrics	DIGITAL_CLK_GEN			TIMING_CONTROL			ARRAY_128_32		
	Acc.	F1	AUC	Acc.	F1	AUC	Acc.	F1	AUC
ParaGraph	0.768	0.847	0.870	0.754	0.841	0.865	0.720	0.776	0.823
DLPL-Cap	0.761	0.841	0.864	0.750	0.839	0.865	0.756	0.832	0.825
CircuitGPS	0.972	0.979	0.992	0.989	0.992	0.998	0.980	0.985	0.999

None of the test datasets are part of the training datasets, and we ensure that all test data is not visible during training and validation. Thus, all results in this work are reported from zero-shot learning.

B. Link Prediction Task

In this subsection, we show the prediction performance of CircuitGPS on the link prediction task. We adapted the two existing MPNN-based models, ParaGraph [18] and DLPL-Cap [19], to our coupling prediction task for comparison purposes. These two models didn't involve any sampling technique or PE, and they directly used the entire graphs and the circuit statistic matrix $\mathbf{X}_C$ as input. We tuned the model hyperparameters to improve their performance. Table V lists the performance comparison. After combining enclosing subgraph sampling and DSPD encoding, CircuitGPS has demonstrated a huge improvement in terms of all performance metrics, improving accuracy by at least 20%.

C. Edge Regression Task

The edge regression task is to predict the exact coupling capacitance of the given node pair. The negative links injected during pre-training were assigned zero capacitance. As capacitance values spanned several magnitudes, we only kept the nets with the capacitance value $1^{-21}F \leq y_C \leq 1^{-15}F$. Then we normalized the statistic attribute matrix $\mathbf{X}_C$ and the capacitance values to $[0, 1]$ to avoid numerical instability. As Table VI lists, CircuitGPS reduces the MAE of capacitance estimation by at least 0.067 compared to other methods.

We also conducted an ablation study of different GPS layer configurations for edge regression on the small training set SSRAM and the test dataset DIGITAL_CLK_GEN. Table VII lists the results. Again, the classic GatedGCN performs strongly, demonstrating Observation 2. Pure Transformer layers have the worst performance due to the limited number of parameters that our computing platform can handle.

D. Extending Study on Node Regression

Beyond link-level tasks, CircuitGPS can also fulfill node-level tasks. We trained CircuitGPS to predict the ground parasitic capacitance on each net/pin. In this task, we didn't

TABLE VI Error Comparison of Different Methods on Edge Regression.

Dataset Metrics	DIGITAL_CLK_GEN			TIMING_CONTROL			ARRAY_128_32		
	MAE↓	RMSE↓	R^2	MAE↓	RMSE↓	R^2	MAE↓	RMSE↓	R^2
ParaGraph	0.153	0.212	0.470	0.154	0.214	0.590	0.181	0.260	0.211
DLPL-Cap	0.160	0.223	0.414	0.157	0.217	0.579	0.176	0.239	0.331
CircuitGPS	0.083	0.130	0.801	0.043	0.097	0.915	0.048	0.120	0.831
CircuitGPS _{head-ft}	0.086	0.125	0.816	0.085	0.131	0.847	0.075	0.120	0.831
CircuitGPS _{all-ft}	0.072	0.120	0.833	0.042	0.093	0.923	0.040	0.074	0.936

TABLE VII Ablation Study of Different GPS Layer Configuration on Edge Regression.

MPNN	Attention	MAE↓	RMSE↓	R^2	Time(s)	#Param.
None	Performer	0.0854	0.1439	0.7563	1437.3	736,871
None	Transformer	0.1051	0.1502	0.7351	2203.6	480,167
GatedGCN	Performer	0.0705	0.12974	0.8019	2667.9	751,311
GatedGCN	Transformer	0.0772	0.1358	0.7831	4765.2	506,703
GatedGCN	None	0.0718	0.1233	0.8212	931.5	723,380

TABLE VIII Error Comparison of Different Methods on Node Regression.

Dataset Metrics	DIGITAL_CLK_GEN			TIMING_CONTROL			ARRAY_128_32		
	MAE↓	RMSE↓	R^2	MAE↓	RMSE↓	R^2	MAE↓	RMSE↓	R^2
ParaGraph	0.101	0.144	0.313	0.112	0.154	0.462	0.114	0.174	0.002
DLPL-Cap	0.137	0.208	0.364	0.096	0.137	0.379	0.097	0.136	0.390
CircuitGPS	0.072	0.104	0.643	0.088	0.132	0.602	0.078	0.101	0.637

involve any negative link injection, and a 2-hop subgraph sampling was adopted to obtain more neighbors around the singular anchor node. The DSPD became identical distances $\mathbf{D}_0 = \mathbf{D}_1$. The architecture remained the same as that in the link-level task. Table VIII shows the predicted errors. CircuitGPS achieves the best performance compared to the other two models in the node regression task. DLPL-Cap has the largest prediction error for the test dataset. This method first learns from a node classification task and then trains a specific regressor for each node class. Therefore, it has data-sensitive characteristics that limit generalization.

Finally, we conducted the circuit simulations using SPICE to validate the predicted capacitance of CircuitGPS (without involving any parasitic resistance). Simulated energy consumption can further demonstrate the effectiveness of CircuitGPS, as this metric is primarily determined by parasitic capacitance in AMS circuits. In Fig. 4, the mean absolute percentage error of three test cases is 14.5%.

V. CONCLUSION

This work introduces CircuitGPS, a few-shot learning framework designed to estimate parasitic effects in AMS circuits. The method incorporates subgraph sampling, a computationally efficient PE, and a customized GT. The robust generalization and transferability of CircuitGPS make it adaptable to a variety of tasks, including link prediction (for coupling effect analysis), edge regression (to estimate coupling capacitance), and node regression (for ground capacitance estimation).

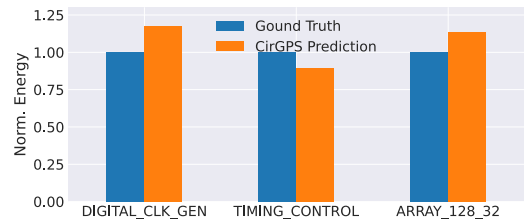


Fig. 4 The simulated energy consumption with ground truth parasitic capacitance vs. predictions of CircuitGPS.

REFERENCES

- [1] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM computing surveys (csur)*, vol. 53, no. 3, pp. 1–34, 2020.
- [2] K. Hakhamaneshi, M. Nassar, M. Phielipp, P. Abbeel, and V. Stojanovic, "Pretraining graph neural networks for few-shot analog circuit modeling and design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 7, pp. 2163–2173, 2022.
- [3] W. Yu and X. Wang, *Advanced field-solver techniques for RC extraction of integrated circuits*. Springer, 2014.
- [4] W. Yu, C. Hu, and W. Zhang, "Variational capacitance extraction of on-chip interconnects based on continuous surface model," in *Proceedings of the 46th Annual Design Automation Conference*, 2009, pp. 758–763.
- [5] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," *Advances in neural information processing systems*, vol. 31, 2018.
- [6] L. Rampásek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a General, Powerful, Scalable Graph Transformer," *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [7] S. Ravi and H. Larochelle, "Optimization as a model for few-shot learning," in *International conference on learning representations*, 2017.
- [8] M. Zhang and Y. Chen, "Weisfeiler-lehman neural machine for link prediction," in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 575–583.
- [9] L. Lü and T. Zhou, "Link prediction in complex networks: A survey," *Physica A: statistical mechanics and its applications*, vol. 390, no. 6, pp. 1150–1170, 2011.
- [10] K. S. Kalyan, A. Rajasekharan, and S. Sangeetha, "Ammus: A survey of transformer-based pretrained models in natural language processing," *arXiv:2108.05542*, 2021.
- [11] K. Han, Y. Wang, H. Chen, X. Chen, J. Guo, Z. Liu, Y. Tang, A. Xiao, C. Xu, Y. Xu *et al.*, "A survey on vision transformer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [12] V. P. Dwivedi and X. Bresson, "A generalization of transformer networks to graphs," *arXiv:2012.09699*, 2020.
- [13] D. Kreuzer, D. Beaini, W. L. Hamilton, V. Létourneau, and P. Tossou, "Rethinking graph transformers with spectral attention," in *Advances in Neural Information Processing Systems*, 2021.
- [14] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do transformers really perform badly for graph representation?" in *Advances in Neural Information Processing Systems*, 2021.
- [15] V. P. Dwivedi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson, "Graph neural networks with learnable structural and positional representations," in *International Conference on Learning Representations*, 2022.
- [16] D. Beaini, S. Passaro, V. Létourneau, W. Hamilton, G. Corso, and P. Liò, "Directional graph networks," in *International Conference on Machine Learning*. PMLR, 2021, pp. 748–758.
- [17] J. Topping, F. Di Giovanni, B. P. Chamberlain, X. Dong, and M. M. Bronstein, "Understanding over-squashing and bottlenecks on graphs via curvature," *arXiv:2111.14522*, 2021.
- [18] H. Ren, G. F. Kokai, W. J. Turner, and T.-S. Ku, "ParaGraph: Layout parasitics and device parameter prediction using graph neural networks," in *Proc. DAC*, 2020, pp. 1–6.
- [19] S. Shen, D. Yang, Y. Xie, C. Pei, B. Yu, and W. Yu, "Deep-learning-based pre-layout parasitic capacitance prediction on sram designs," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, 2024, pp. 440–445.
- [20] K. M. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Q. Davis, A. Mohiuddin, L. Kaiser *et al.*, "Rethinking attention with performers," in *International Conference on Learning Representations*, 2020.
- [21] V. P. Dwivedi, C. K. Joshi, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking graph neural networks," *arXiv:2003.00982*, 2020.
- [22] M. Black, Z. Wan, G. Mishne, A. Nayyeri, and Y. Wang, "Comparing graph transformers via positional encodings," *arXiv preprint arXiv:2402.14202*, 2024.
- [23] S. Shen, T. Shao, X. Shang, Y. Guo, M. Ling, J. Yang, and L. Shi, "TS cache: A fast cache with timing-speculation mechanism under low supply voltages," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 1, pp. 252–262, 2019.
- [24] L. Shi, B. Hu, D. Zhao, J. He, Z. Zhang, and J. Zhou, "Structural information enhanced graph representation for link prediction," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 13, 2024, pp. 14 964–14 972.
- [25] X. Bresson and T. Laurent, "Residual gated graph convnets," *arXiv preprint arXiv:1711.07553*, 2017.
- [26] Y. Luo, L. Shi, and X.-M. Wu, "Classic gnns are strong baselines: Reassessing gnns for node classification," *arXiv preprint arXiv:2406.08993*, 2024.
- [27] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [28] J. You, R. Ying, and J. Leskovec, "Design space for graph neural networks," in *Advances in Neural Information Processing Systems*, 2020.
- [29] S. Shen, H. Xu, Y. Zhou, M. Ling, and W. Yu, "Ultra8t: A sub-threshold 8t sram with leakage detection," *Integration*, vol. 98, p. 102233, 2024.
- [30] J. Yang, Y. Kong, Z. Wang, Y. Liu, B. Wang, S. Yin, and L. Shi, "24.4 sandwich-RAM: An energy-efficient in-memory BWN architecture with pulse-width modulation," in *Proc. Int. Solid-State Circuits Conf. (ISSCC)*, 2019, pp. 394–396.